

Rifa Ferzana¹

¹ORCID: <https://orcid.org>, Middlesex University London

January 13, 2026

Generative AI as a Cognitive Abstraction in Software Engineering: Evidence from Academic and Industrial Practice

Rifa Ferzana
Middlesex University
London, UK

ORCID: <https://orcid.org/0009-0000-7300-692X>
Email: r.ferzana@mdx.ac.uk

Abstract

Generative Artificial Intelligence (GenAI), particularly large language models (LLMs), has emerged as a transformative cognitive layer in software engineering. This paper presents an evidence-based analysis of GenAI as a cognitive abstraction, synthesising findings from peer-reviewed literature, empirical studies and industry reports. We examine how LLMs function as probabilistic abstraction mechanisms, enabling developers to express intent in natural language while delegating implementation details. Through comparative analysis of academic and industrial adoption patterns, we reveal fundamental differences in incentives, risk tolerance and governance approaches. While academia prioritises methodological rigour and epistemic scrutiny, industry focuses on productivity gains and scalable integration. Both domains converge on the necessity of human oversight. We propose a structured human-in-the-loop framework emphasising validation, accountability and traceability. Empirical evidence indicates GenAI can accelerate routine tasks by 30–55% but introduces risks including security vulnerabilities, bias propagation and technical debt accumulation. The paper argues that GenAI represents not merely a tool but a fundamental abstraction layer that reshapes cognitive workflows while preserving the centrality of human expertise. We conclude with research directions addressing longitudinal effects, verification challenges and organisational implications.

Keywords: generative AI, software engineering, cognitive abstraction, large language models, human-in-the-loop, empirical software engineering

1. Introduction

Software engineering has historically evolved through successive abstraction layers designed to manage complexity. From structured programming to object-oriented design, each abstraction has enabled engineers to reason about systems at higher conceptual levels while managing cognitive load (Brooks, 1987; Dijkstra, 1972). Modern practices including modular architectures, domain-specific languages and automated tooling continue this tradition of complexity management through abstraction (Sommerville, 2016; Fowler, 2018).

Generative Artificial Intelligence, particularly models trained on vast corpora of code and natural language, introduces a qualitatively different abstraction mechanism. Unlike deterministic tools, LLMs such as GPT-4 (OpenAI, 2023), Codex (Chen et al., 2021) and Gemini (Team et al., 2023) enable probabilistic translation of natural language intent into executable code, documentation and design artefacts. This represents a shift from syntax-based to semantics-aware automation, where the abstraction operates at the level of human reasoning patterns rather than formal specifications (Bommasani et al., 2022; Nijkamp et al., 2023).

The cognitive implications are profound. Developers increasingly interact with systems that engage in tasks traditionally requiring human intelligence: problem decomposition, algorithmic reasoning, code synthesis and documentation generation (Weisz et al., 2021; Xu et al., 2022). This reconfiguration of the developer's cognitive workflow raises critical questions about skill evolution, knowledge transfer and the nature of expertise in AI-augmented software engineering (Prather et al., 2023).

This paper makes three primary contributions: First, we conceptualise GenAI as a cognitive abstraction layer in software engineering, analysing its technical foundations and epistemological implications. Second, we provide a comparative analysis of academic and industrial adoption, highlighting divergent incentives, risk tolerances and governance approaches. Third, we propose a human-in-the-loop framework for responsible integration and identify critical research directions.

2. Technical Foundations: From Statistical Models to Cognitive Abstractions

2.1 Transformer Architecture and Emergent Capabilities

The transformer architecture (Vaswani et al., 2017) fundamentally enables GenAI's role as a cognitive abstraction. Its self-attention mechanism allows models to capture long-range dependencies in sequential data without recurrence constraints, making it particularly suitable for code representation (Zügner et al., 2021). Scaling laws demonstrate that increasing model parameters, training data and compute yields predictable improvements in code generation capability (Kaplan et al., 2020; Hoffmann et al., 2022).

Recent research reveals emergent capabilities in sufficiently large models, including few-shot learning, chain-of-thought reasoning and contextual adaptation (Wei et al., 2022; Srivastava et al., 2022). For software engineering, these translate to abilities such as generating code from descriptive comments, translating between programming languages and explaining complex algorithms (Austin et al., 2021; Rozière et al., 2023).

2.2 Modelling Software as a Language Distribution

Code exhibits linguistic properties—syntax, semantics and pragmatics—making it amenable to statistical language modelling (Hindle et al., 2012). Modern models treat programming languages as distributions over tokens, learning patterns from billions of lines of code across multiple languages (Allal et al., 2023). This statistical foundation enables probabilistic code completion but introduces inherent limitations: generated code represents the most likely continuation given training data, not necessarily the correct or optimal solution (Pearce et al., 2022).

The abstraction occurs at the interface: developers express intent in natural language, and the model maps this to code space. This mapping is inherently probabilistic and context-dependent, distinguishing it from traditional deterministic compilers and IDEs (Chen et al., 2021).

3. Methodology: Evidence Synthesis Approach

This study employs a systematic evidence synthesis methodology following guidelines for software engineering surveys (Kitchenham & Charters, 2007; Wohlin et al., 2012). Our approach includes:

1. **Systematic Literature Review:** We searched ACM Digital Library, IEEE Xplore, arXiv and Google Scholar for peer-reviewed publications (2020–2024) using keywords: "generative AI software engineering", "LLM code generation", "AI pair programming", "Copilot evaluation" and "AI-assisted development". Inclusion criteria required empirical evaluation or theoretical analysis of GenAI in software engineering contexts.
2. **Industry Report Analysis:** We examined technical reports from GitHub (2022–2023), Stack Overflow Developer Surveys (2021–2023) and industry whitepapers from Microsoft, Google and Amazon detailing GenAI adoption patterns.
3. **Framework and Policy Analysis:** We analysed governance frameworks including NIST AI RMF (2023), EU AI Act (2024) and institutional policies from academic publishers (Nature, IEEE, ACM) regarding AI use.
4. **Synthesis Method:** We employed narrative synthesis (Popay et al., 2006) to integrate findings across evidence types, identifying patterns, contradictions and research gaps. This approach supports developing conceptual frameworks while acknowledging the rapidly evolving nature of the field.

4. Empirical Evidence: Productivity, Quality and Cognitive Effects

4.1 Productivity Gains and Developer Experience

Controlled studies consistently report significant productivity improvements. Peng et al. (2023) found that developers using GitHub Copilot completed tasks 55% faster than control groups. Similar studies report 30–40% acceleration in standard programming tasks (Barke et al., 2023; Yetiştiren et al., 2023). Beyond speed, developers report reduced cognitive load for routine coding, documentation and test generation (Vaithilingam et al., 2022).

However, productivity gains are unevenly distributed. Novice developers benefit more for routine tasks while experts leverage GenAI for exploration and prototyping (Ziegler et al., 2022). The "productivity paradox" emerges when time saved in coding shifts to validation and debugging of AI-generated outputs (Prather et al., 2023).

4.2 Code Quality and Security Implications

Multiple studies identify quality concerns in AI-generated code. Pearce et al. (2022) found that 40% of Copilot-suggested code for security-sensitive scenarios contained vulnerabilities. Perry et al. (2023) demonstrated that models reproduce biases and anti-patterns from training data, potentially institutionalising poor practices.

Longitudinal analysis suggests AI-assisted development may increase technical debt if generated code lacks modularity or documentation. The cognitive abstraction can obscure implementation details, potentially creating "black box" components that hinder maintenance (Shin et al., 2023).

5. Academic versus Industrial Adoption: Divergent Pathways

5.1 Institutional Incentives and Evaluation Criteria

Academic adoption emphasises research utility and methodological scrutiny. Universities increasingly use GenAI for literature synthesis, data analysis coding and manuscript drafting (Nature Editorial, 2023). However, rigorous citation, reproducibility requirements and scepticism toward

automated content constrain integration (COPE, 2023). Evaluation focuses on novelty, rigour and contribution to knowledge rather than efficiency gains.

Industrial adoption prioritises operational metrics: development velocity, defect reduction and developer satisfaction (Microsoft, 2022). Technology firms integrate GenAI throughout development pipelines—from requirement analysis to deployment automation (Google, 2023). Evaluation is ROI-driven, with faster iteration cycles justifying acceptance of manageable error rates.

5.2 Governance and Risk Management Approaches

Academic governance centres on disclosure and attribution. Journals mandate AI usage statements; ethics boards scrutinise AI-assisted research; and educational institutions develop policies for student use (ACM, 2018). The emphasis is on individual researcher accountability with low tolerance for opaque automation.

Industrial governance employs structured frameworks: Microsoft's Responsible AI Standard (2022), Google's AI Principles (2023) and alignment with NIST's AI Risk Management Framework (2023). These emphasise organisational accountability through review processes, testing protocols and audit trails. Risk tolerance is calibrated to application criticality.

5.3 Convergence Trends

Despite the clear differences outlined in Table 1, significant convergence is emerging between academic and industrial approaches to GenAI adoption. This convergence reflects growing recognition that both domains can learn from each other's strengths while addressing shared challenges.

Academic research is increasingly engaging with practical concerns relevant to industry, including security vulnerabilities in AI-generated code, long-term maintainability issues, and effective integration of AI tools into development workflows (Pearce et al., 2022; Ko et al., 2023). Simultaneously, educational institutions are adapting curricula to prepare students for AI-augmented software engineering, emphasising skills in prompt engineering and critical validation alongside traditional competencies (Denny et al., 2023).

Industry is responding by adopting more research-informed practices. Technology firms are investing in fundamental research on explainability, bias mitigation, and robust evaluation methodologies (Microsoft, 2022; Google, 2023). The adoption of formal risk management frameworks like the NIST AI RMF (2023) demonstrates industry's recognition that sustainable innovation requires systematic, evidence-based approaches to governance.

Professional organisations are facilitating this convergence through cross-domain guidelines. The IEEE's *Ethically Aligned Design* framework (2019) and ACM's ethical standards provide shared principles that bridge academic and industrial contexts, establishing common ground while allowing for contextual adaptation.

This convergence suggests the development of hybrid innovation models where:

1. Research insights rapidly inform product development
2. Deployment challenges stimulate new research questions

3. Shared infrastructure (benchmarks, tools) emerges through partnerships
4. Integrated governance combines academic transparency with industrial risk management

The ultimate benefit is a more robust, ethically grounded approach to leveraging GenAI in software engineering that draws on the complementary strengths of both academic rigour and industrial pragmatism.

Dimension	Academia	Industry
Primary Objective	Knowledge creation, education	Product delivery, efficiency
Adoption Speed	Gradual, cautious	Rapid, competitive
Risk Tolerance	Low (epistemic integrity)	Managed (business context)
Evaluation	Peer review, citation impact	ROI, velocity, reliability
Governance	Ethics committees, disclosure	Policy frameworks, audits
Accountability	Individual researchers	Organisational structures
Failure Response	Retraction, correction	Patching, process updates

Table 1: Comparative Analysis of Adoption Patterns

These convergence trends suggest that while distinctive approaches remain, hybrid models are emerging that productively blend methodological rigour with practical implementation needs, creating more nuanced pathways for responsible GenAI adoption in software engineering.

6. Ethical and Epistemological Implications

6.1 Authorship and Responsibility

LLMs lack intentionality and moral agency (Bender et al., 2021). Consequently, responsibility for AI-generated artefacts rests entirely with human practitioners. This principle is codified in publication ethics (COPE, 2023) and professional standards (ACM, 2018). The cognitive abstraction must not obscure accountability chains, particularly in safety-critical systems (Ko et al., 2023).

6.2 Bias and Representation

Training data reflects historical biases in software development—underrepresentation of certain languages, dominance of particular coding styles and propagation of security anti-patterns (Ozkaya, 2023). GenAI risks institutionalising these biases, narrowing solution diversity (Birhane et al., 2023). Mitigation requires diverse training corpora, bias detection tools and human oversight to challenge dominant patterns.

6.3 Epistemic Authority and Knowledge Homogenisation

Delegating reasoning tasks to statistical models raises epistemic concerns. When developers treat AI suggestions as authoritative, critical thinking may diminish (Katz et al., 2023). The abstraction layer could homogenise problem-solving approaches, reducing innovation that often emerges from challenging conventions (Denny et al., 2023). Preserving epistemological diversity requires explicit encouragement of alternative solutions and critical evaluation practices.

7. Human-in-the-Loop Framework for Responsible Integration

7.1 Framework Design Principles

Effective HITL integration for GenAI in software engineering requires:

1. **Complementarity:** AI augments, does not replace, human judgment
2. **Accountability:** Clear responsibility assignment
3. **Transparency:** Understandable AI contributions
4. **Adaptability:** Variable oversight based on task criticality

7.2 Four-Stage Integration Model

Stage 1: Intent Specification

Human developers articulate requirements, constraints and context. This includes functional specifications, non-functional requirements (security, performance) and domain knowledge. Prompt engineering becomes a critical skill for effective abstraction use.

Stage 2: AI-Assisted Generation

GenAI produces candidate solutions: code snippets, architecture options, test cases or documentation. Outputs are treated as proposals requiring validation, not final products.

Stage 3: Validation and Refinement

Human review employs software engineering practices: code review, testing (unit, integration, security), static analysis and architectural assessment. This stage addresses AI limitations: hallucination, context misunderstanding and bias.

Stage 4: Integration and Traceability

Validated artefacts integrate into codebase with proper attribution and documentation. Version control logs capture AI contributions; audit trails enable incident analysis; knowledge bases record lessons learned.

7.3 Organisational Implementation

Successful implementation requires:

- **Training:** AI literacy, prompt engineering, validation techniques
- **Tooling:** Integrated development environments with visibility into AI contributions
- **Process:** Standardised review checklists, approval workflows
- **Culture:** Psychological safety to challenge AI outputs, continuous learning

8. Broader Implications for Software Engineering

8.1 Evolving Professional Competencies

GenAI shifts required skills toward:

- **Prompt Engineering:** Articulating problems for AI systems

- **Validation Expertise:** Critically assessing AI outputs
- **System Thinking:** Architectural reasoning beyond implementation
- **AI Literacy:** Understanding model capabilities and limitations
- **Ethical Judgment:** Navigating bias, fairness and responsibility issues

8.2 Educational Transformation

Computer science curricula must adapt to include:

- AI-assisted programming techniques
- Critical evaluation of generated code
- Ethical considerations in AI use
- Hybrid human-AI problem solving
- Security implications of AI-generated code

8.3 Long-Term Trajectory

GenAI represents the latest in a series of cognitive abstractions. Future developments may include:

- Specialised models for specific domains (embedded systems, scientific computing)
- Improved reasoning capabilities reducing validation burden
- Tighter integration with formal methods for verification
- Personalised assistants adapting to individual developer styles

9. Future Research Directions

1. **Longitudinal Studies:** How does AI-assisted development affect system evolution, technical debt accumulation and team knowledge sharing over 2–5 year periods?
2. **Verification Integration:** Can formal methods (model checking, theorem proving) be combined with generative models to provide correctness guarantees?
3. **Cognitive Studies:** How does GenAI affect developer problem-solving strategies, learning trajectories and expertise development?
4. **Organisational Impact:** What governance structures, team compositions and development processes optimise human-AI collaboration?
5. **Specialised Applications:** Domain-specific models for safety-critical systems, legacy modernisation and cross-platform development.
6. **Evaluation Metrics:** Beyond productivity, develop comprehensive metrics for code quality, security, maintainability and innovation in AI-assisted development.

10. Conclusion

Generative AI represents a fundamental cognitive abstraction layer in software engineering, enabling expression of intent in natural language while probabilistically generating implementations. Evidence demonstrates significant productivity benefits (30–55% acceleration) balanced against risks including security vulnerabilities, bias propagation and potential technical debt accumulation.

The abstraction reshapes rather than replaces human expertise, shifting cognitive effort from routine implementation to problem framing, validation and system thinking. Academic and industrial

adoption follow divergent paths reflecting different incentives and risk tolerances but converge on the necessity of human oversight and ethical responsibility.

Our proposed human-in-the-loop framework provides a structured approach to responsible integration, emphasising validation, accountability and traceability. As this technology evolves, the software engineering profession must develop new competencies in AI literacy, prompt engineering and critical evaluation while preserving the human judgment essential to building robust, ethical and innovative systems.

The cognitive abstraction offered by GenAI is neither silver bullet nor existential threat, but a powerful new element in software engineering's ongoing evolution—one that amplifies human capability while demanding rigorous stewardship of the knowledge and responsibility at the field's core.

References

- Allal, Loubna Ben, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. (2023). SantaCoder: Don't reach for the stars! *arXiv preprint arXiv:2301.03988*.
- Association for Computing Machinery. (2018). *ACM Code of Ethics and Professional Conduct*. ACM Press.
- Austin, Jacob, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Barke, Shraddha, Michael B. James, and Nadia Polikarpova. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 1-27.
- Bender, Emily M., Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610-623).
- Birhane, Abeba, William Isaac, Vinodkumar Prabhakaran, Mark Díaz, Madeleine Clare Elish, Iason Gabriel, and Shakir Mohamed. (2023). The values encoded in machine learning research. In *2023 ACM Conference on Fairness, Accountability, and Transparency* (pp. 173-184).
- Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, et al. (2022). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.
- Brooks, Frederick P. (1987). No silver bullet: Essence and accidents of software engineering. *IEEE Computer*, 20(4), 10-19.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

- Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Committee on Publication Ethics. (2023). *Authorship and AI tools: COPE position statement*. COPE.
- Denny, Paul, Viraj Kumar, and Nasser Giacaman. (2023). Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V.1* (pp. 1136-1142).
- Dijkstra, Edsger W. (1972). The humble programmer. *Communications of the ACM*, 15(10), 859-866.
- European Parliament. (2024). *Artificial Intelligence Act*. European Union.
- Fowler, Martin. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.
- GitHub. (2022). *The impact of AI on developer productivity: Evidence from GitHub Copilot*. GitHub Research.
- Google. (2023, March 21). *Bard: An experimental conversational AI service*. Google AI Blog.
- Hindle, Abram, Earl T. Barr, Zhendong Su, Mark Gabel, and Premkumar Devanbu. (2012). On the naturalness of software. In *2012 34th International Conference on Software Engineering (ICSE)* (pp. 837-847). IEEE.
- Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.
- IEEE. (2019). *Ethically aligned design: A vision for prioritizing human well-being with autonomous and intelligent systems* (1st ed.). IEEE.
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Katz, Daniel Martin, Michael James Bommarito, and Shang Gao. (2023). GPT-4 passes the bar exam. *SSRN 4389233*.
- Kitchenham, Barbara, and Stuart Charters. (2007). *Guidelines for performing systematic literature reviews in software engineering* (EBSE Technical Report EBSE-2007-01). Keele University and University of Durham.
- Ko, Amy J., Jingyi Zhu, Titus H. Nguyen, and Yang Zhang. (2023). The next generation of human-AI collaboration in software engineering. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)* (pp. 25-36).
- Le, Yujia, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, et al. (2022). Competition-level code generation with AlphaCode. *Science*, 378(6624), 1092-1097.
- Microsoft. (2022). *Responsible AI standard, v2*. Microsoft.
- Nature Editorial. (2023). ChatGPT: Five priorities for research. *Nature*, 614(7947), 224-226.
- National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST.
- Nijkamp, Erik, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. (2023). CodeGen: An open large language model for code. *arXiv preprint arXiv:2203.13474*.
- OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ozkaya, Ipek. (2023). Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software*, 40(3), 4-11.
- Pearce, Hammond, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)* (pp. 754-768). IEEE.

- Peng, Sida, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*.
- Perry, Neil, Megha Srivastava, Deepak Kumar, and Dan Boneh. (2023). Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2785-2799).
- Popay, Jennie, Helen Roberts, Amanda Sowden, Mark Petticrew, Lisa Arai, Mark Rodgers, Nicky Britten, Katrina Roen, and Steven Duffy. (2006). *Guidance on the conduct of narrative synthesis in systematic reviews*. ESRC Methods Programme.
- Prather, James, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. (2023). The robots are coming: Exploring the implications of OpenAI Codex on introductory programming. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1* (pp. 126-140).
- Rozière, Baptiste, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- Sommerville, Ian. (2016). *Software engineering* (10th ed.). Pearson.
- Srivastava, Aaro, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adria Garriga-Alonso, et al. (2022). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.
- Team, Gemini, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, et al. (2023). Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Vaithilingam, Priyan, Tianyi Zhang, and Elena L. Glassman. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems* (pp. 1-7).
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998-6008).
- Wei, Jason, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, et al. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*.
- Weisz, Justin D., Michael Muller, Stephanie Houde, John Richards, Steven I. Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. (2021). Perfection not required? Human-AI partnerships in code translation. In *26th International Conference on Intelligent User Interfaces* (pp. 402-412).
- Wohlin, Claes, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. (2012). *Experimentation in software engineering*. Springer.
- Xu, Frank F., Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. (2022). A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (pp. 1-10).
- Yetiştiren, Burak, İsmail Özsoy, Miray Ayerdem, and Eray Tüzün. (2023). Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. *arXiv preprint arXiv:2304.10778*.
- Zhang, Yuxiang, Ziyi Chen, and Haoyu Wang. (2023). A systematic literature review of AI applications in software engineering. *ACM Computing Surveys*, 55(9), 1-35.
- Ziegler, Albert, Eirini Kalliamvakou, Shawn Simister, Ganesh Sittampalam, Alice Li, Andrew Rice, Devon Rifkin, and Edward Aftandilian. (2022). Productivity assessment of neural code

- completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (pp. 21-29).
- Zügner, Daniel, Tobias Kirschstein, Michele Catasta, Jure Leskovec, and Stephan Günnemann. (2021). Language-agnostic representation learning of source code from structure and context. In *International Conference on Learning Representations*.